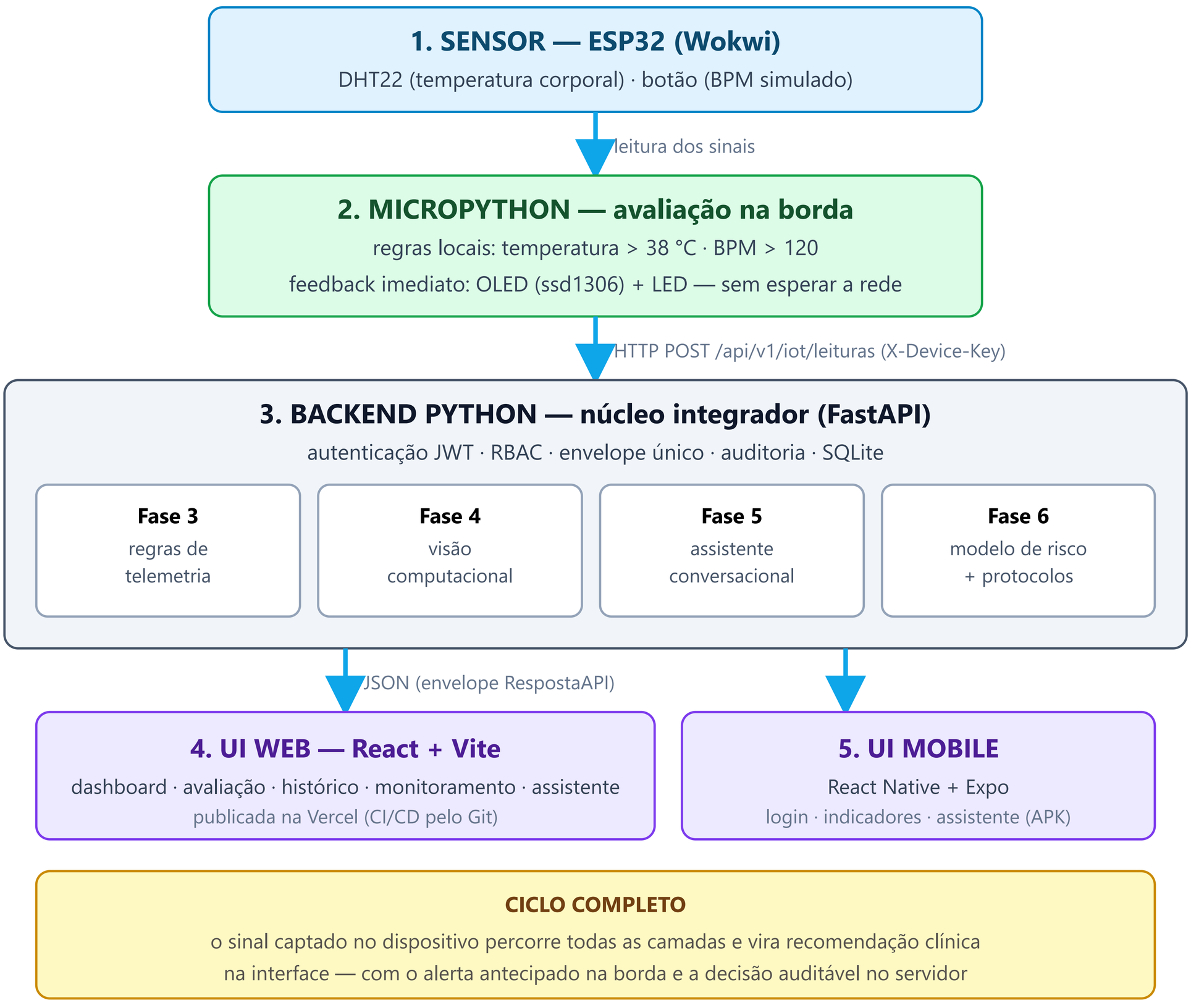
CardioIA — Fase 7

FIAP
Renan de Oliveira Mendes — RM563145

Relatório técnico · MVP final e funcional da plataforma
Arquitetura final: Sensor → MicroPython → Backend Python → APIs de IA → UI

Aviso: os dados do projeto são **sintéticos** e não há validação clínica. Nenhum resultado constitui diagnóstico, prescrição ou recomendação médica. Este documento descreve uma solução acadêmica.

1. Arquitetura final



2. Componentes e tecnologias

Camada	Tecnologia	Responsabilidade
Sensor	ESP32 simulado no Wokwi; DHT22; botão; OLED; LED	Captura de temperatura corporal e de frequência cardíaca simulada
Borda	**MicroPython** (`IoT/main.py`)	Avalia os sinais no próprio dispositivo e sinaliza em OLED e LED sem depender da rede
Integrador	**Python / FastAPI** (`backend/`)	Conecta as interfaces aos motores de IA; aplica autenticação, autorização, envelope único, auditoria e persistência
IA	Fase 6 (modelo de risco), Fase 5 (assistente), Fase 4 (visão), Fase 3 (regras de telemetria)	Produz a probabilidade de risco, os protocolos aplicáveis e a resposta conversacional
UI Web	React + Vite	Dashboard, avaliação de risco, histórico, monitoramento e assistente
UI Mobile	React Native + Expo	Login, indicadores cardíacos e assistente, distribuído como APK

2.1 O backend como núcleo integrador

O backend é o ponto único de integração. Cada módulo das fases anteriores é acessado por uma camada dedicada em `backend/app/integracao/` , o que mantém as fases anteriores **intactas** e permite evoluí-las de forma independente:

Módulo	Como é acessado	Por quê
Fase 6 — pipeline e modelo	Import controlado, com `sys.path` e carga tardia	É código Python local; a carga do artefato é adiada para não pesar o boot
Fase 5 — assistente	Provedor local (padrão) e adaptador opcional do provedor externo	O provedor local responde da base de conhecimento do projeto, sem dependência de rede ou credencial
Fase 4 — visão	Chamada HTTP a um serviço em processo separado	Isola o TensorFlow do backend central; serviço fora do ar devolve `502`
Fase 3 — regras de telemetria	Reimplementação fiel, verificada contra o código de origem	O serviço legado em Flask foi desativado; as regras vivem em `app/regras_iot.py`

3. Fluxo de dados

1. **Captura.** O ESP32 lê o DHT22 e conta batimentos por toque no botão. A frequência é derivada do intervalo entre toques — a mesma conta que um oxímetro faz.
2. **Avaliação na borda.** O MicroPython aplica as regras (`temperatura > 38` , `bpm > 120`) e reage na hora: o OLED mostra os sinais e o alerta, o LED acende. Nenhuma decisão depende da rede.
3. **Transporte.** A leitura vai por `HTTP POST` para `/api/v1/iot/leituras` , autenticada pelo cabeçalho `X-Device-Key` .
4. **Ingestão.** O backend carimba `recebido_em` , reavalia os sinais com as mesmas regras, persiste a leitura e devolve o alerta em `avisos` .
5. **Recomendação.** Ao registrar uma avaliação, o backend executa o pipeline da Fase 6: o modelo calcula a probabilidade de pico de risco e o sistema seleciona os protocolos aplicáveis, com a trilha de execução registrada.
6. **Entrega.** Web e mobile consomem o mesmo contrato e exibem o risco, os indicadores e a orientação — separando o que é dado de dispositivo, resultado preditivo e conteúdo do assistente.

4. Segurança e qualidade

4.1 Controle de acesso

Papel	Pode
`MEDICO`	ler e escrever dado clínico; assistente; visão
`ADMIN`	apenas leitura clínica
`OPERADOR`	telemetria e assistente

A autorização é decidida **no servidor** ; as interfaces apenas refletem o que ele permite. O papel gravado no token é reconferido no banco a cada requisição — mudar o papel de alguém invalida os tokens já emitidos.

4.2 Proteção de credenciais

- Senhas apenas como **hash bcrypt** (custo 12).
- Refresh tokens guardados como **hash SHA-256** , com rotação a cada uso.
- Segredos **somente** por variável de ambiente; o serviço não sobe sem eles.
- Credenciais das fases anteriores **sanitizadas** antes da publicação do repositório.

4.3 Contrato único de resposta

Toda resposta — sucesso ou falha — usa o mesmo envelope, com `status` , `dados` , `avisos` , `limitacoes` e `erros` . O erro carrega um **código estável** , não apenas texto livre, o que permite à interface reagir de forma precisa: `NAO_AUTENTICADO` , `NAO_AUTORIZADO` , `NAO_CONFIGURADO` , `ERRO_DE_SERVICO_DE_VISAO` .

5. Decisões de interface e integridade do dado

Duas regras governam as três interfaces. Elas não são preferência estética: derivam do contrato e do risco de um número ser lido como algo que ele não é.

5.1 O dado primeiro, a ressalva a um clique

Avisos, limitações declaradas, narrativa do pipeline, justificativas e trilha de execução ficam **recolhidos** — presentes, a um clique, mas fora do caminho de leitura. O que permanece visível é o que muda uma decisão.

Elemento	Por que aparece assim
Avisos e limitações declaradas	O contrato exige que **existam** na resposta; não que ocupem a tela
Narrativa, justificativas e trilha de execução	Texto de auditoria, não de leitura clínica
Protocolos aplicáveis e indicadores deterministas	**Visíveis** : são o conteúdo acionável da resposta

5.2 Nenhum dado sintético se passa por medição

- **Ausência não é zero.** Campo não informado aparece como `--` ; o backend nunca preenche valor padrão em nome do usuário.
- **A origem do dado é rotulada.** Uma leitura chega como `rest` , `mqtt` ou `simulado` — e a interface exibe esse rótulo. Uma leitura semeada para demonstração não pode ser lida como medição de dispositivo real.
- **O que o modelo não devolve não é exibido.** A API entrega os *nomes* das features usadas, não os valores; as barras só aparecem para quem acabou de informar os valores. Preencher o resto seria apresentar dado que o backend não devolveu.

A mesma regra vale para os alertas: o alerta das regras de telemetria é **derivado** e não tem coluna no banco. Ele aparece na resposta da ingestão; no histórico, a interface mostra os sinais brutos e declara a diferença, em vez de recalcular o limiar no cliente — o que criaria uma segunda fonte de verdade clínica.

6. Validação executada

Verificação	Resultado	O que cobre
Suíte automatizada do backend	**659 testes aprovados**	Contratos, persistência, autenticação, integração com as fases 3 a 6, CORS
Integração ponta a ponta (smoke)	**75 verificações, 0 falhas**	Fluxo real nos três papéis, contra o backend em execução — 33 médico, 23 operador e 19 admin
Pipeline da Fase 6 pelo cliente	probabilidade **0,99** · `pico_de_risco`	O modelo é carregado e executado de fato, não simulado
Regra da Fase 3 na ingestão	`FEBRE DETECTADA`	Leitura de 38,9 °C aciona o alerta correto
Indisponibilidade de serviço externo	`502` com mensagem acionável	A interface informa a causa em vez de falhar em silêncio
Empacotamento da interface web	46 módulos, 285 KB (87 KB comprimido)	Todas as telas e imports resolvem
Análise estática do frontend	0 erros, 0 avisos	Convenções e boas práticas de React

7. Limitações declaradas

- **Dados sintéticos, sem validação clínica.** O modelo foi treinado em base acadêmica; a probabilidade é do *episódio assistencial* , não um prognóstico individual.
- **Temperatura é a feature de menor confiança.** O gerador de treino usou distribuição de temperatura ambiente, enquanto a feature representa temperatura corporal — o valor cai na cauda da distribuição. A interface marca isso.
- **A telemetria não é vinculada a paciente.** O contrato associa a leitura ao *dispositivo* ; as interfaces rotulam os sinais vitais como tal.
- **O alerta das regras não é persistido.** É derivado dos sinais e devolvido em `avisos` na ingestão.
- **O assistente orienta, não diagnostica.** As respostas encaminham para atendimento de emergência diante de sinais de alerta e não substituem avaliação profissional.
- **A frequência cardíaca é simulada por botão** , conforme o escopo do trabalho — não há sensor de pulso no circuito.

8. Conclusão

A plataforma integra os módulos das fases anteriores em um ecossistema funcional, no qual um sinal captado no dispositivo percorre avaliação de borda, ingestão, processamento por modelos de IA e chega às interfaces web e mobile como recomendação clínica. A arquitetura foi desenhada para **não depender de credenciais de terceiros** : cada serviço externo que normalmente exigiria conta tem um equivalente dentro do próprio projeto, o que mantém a solução executável e reproduzível.